

Manual de usuario – Pipeline de scoring mensual

Este manual está dirigido a la persona que ejecuta el proceso de **scoring (inferencia)** cada mes para obtener puntajes de riesgo de fraude/anomalía en el consumo de agua. No es necesario conocer el código; solo editar un archivo de configuración y ejecutar dos comandos (o uno, si no hay datos nuevos).

1. Qué hace el proceso

- **ETL:** Toma los archivos de inspecciones y consumo que usted coloca en `data/raw/`, los limpia y guarda en `data/interim/`.
- **Inferencia (scoring):** Usa el modelo ya entrenado y el mes que usted indique para calcular un puntaje de riesgo por contrato. El resultado es un archivo CSV con los scores.

Cada mes usted solo debe **indicar el mes a predecir** y ejecutar el proceso. El resto de la configuración suele no cambiar.

2. Requisitos previos

- **Entorno Python** ya configurado en su máquina (entorno virtual activado, dependencias instaladas con `pip install -r requirements.txt`). Si no lo tiene, solicite apoyo al equipo técnico o consulte [docs/setup.md](#) si existe.
 - **Datos de entrada:** En la carpeta `data/raw/` deben estar los archivos del mes según el formato esperado:
 - `data/raw/inspecciones/inspecciones_AAAA_MM.xlsx`
 - `data/raw/consumo/consumo_AAAA_MM.xlsx` (sustituya AAAA por el año y MM por el mes, por ejemplo 2026 y 03).
 - **Modelo en uso:** En la carpeta `models/` deben existir los archivos `lgbm_model.pkl` y `features.pkl` (proporcionados o generados por el equipo que entrena el modelo).
-

3. Paso obligatorio cada mes: indicar el mes a predecir

Antes de ejecutar la inferencia, debe editar el archivo de configuración para indicar **qué mes se va a predecir**.

1. Abra el archivo `config/config.yaml` (en la raíz del proyecto).
2. Al inicio del archivo verá la sección **inference** y el parámetro **cutoff**.
3. Cambie el valor de **cutoff** al mes que desea predecir, en formato **YYYY-MM-DD** (año-mes-día; el día puede ser 01).

Ejemplo: Para predecir junio de 2025:

```
inference:
  cutoff: "2025-06-01"
  cant_periodos: 12
```

```
contratos_list: null
columns_filter: null
```

- Use siempre comillas alrededor de la fecha.
- El formato debe ser exactamente **YYYY-MM-DD** (por ejemplo "2025-06-01" para junio de 2025).
- No modifique el resto del archivo a menos que le indiquen lo contrario.

Parámetro opcional: `columns_filter` (filtrar por categoría, ciclo, etc.)

Si desea procesar solo un subconjunto de contratos (por ejemplo, solo categoría "Industrial" o solo ciertos ciclos), puede usar **`columns_filter`**:

```
inference:
  cutoff: "2025-06-01"
  columns_filter:
    categoria: ["Industrial"]
```

- **`columns_filter: null`** → procesa todos los contratos con consumo en la ventana (comportamiento por defecto).
- **`columns_filter: { categoria: ["Industrial"] }`** → solo contratos de categoría Industrial.
- **`columns_filter: { ciclo: ["14", "15"] }`** → solo contratos de ciclos 14 y 15.

Este filtro se aplica antes de calcular las variables de series de tiempo, lo que acelera el proceso si solo necesita un segmento.

Parámetro opcional: `output_columns` (columnas en el CSV de salida)

En `inference.output_columns` puede indicar qué columnas adicionales desea en el archivo de resultados además de **contrato** y **score** (por ejemplo categoría, estrato, municipio, localidad). Si pone **null** o no existe la opción, el CSV solo tendrá contrato y score. El equipo técnico puede ajustar la lista según lo que necesite para priorizar inspecciones.

4. Cómo ejecutar (desde la raíz del proyecto)

Abra una terminal y sitúese en la **raíz del proyecto** (la carpeta que contiene `config/`, `scripts/`, `data/`, etc.).

4.1. ETL (solo si hay datos nuevos)

Si cargó archivos nuevos en `data/raw/` (inspecciones o consumo del mes), ejecute primero el ETL:

```
python scripts/run_etl.py
```

Si desea reprocesar todos los meses desde cero (no solo los nuevos):

```
python scripts/run_etl.py --overwrite
```

Si no hay archivos nuevos en `data/raw/`, puede **saltar este paso** y pasar directo a la inferencia.

4.2. Inferencia (scoring)

Ejecute:

```
python scripts/run_inference.py
```

El script leerá el **cutoff** que configuró en `config/config.yaml` y generará el archivo de scores para ese mes.

5. Dónde está el resultado

Al finalizar la inferencia sin errores, el archivo de resultados se guarda en:

`data/predictions/scores_<CUTOFF>_<timestamp>.csv`

- **<CUTOFF>** es la fecha que puso en `inference.cutoff` (por ejemplo `2025-06-01`).
- **<timestamp>** es la fecha y hora de esa ejecución (formato `YYYYMMDD_HHMMSS`), para no sobrescribir ejecuciones anteriores.

Ejemplo: `scores_2025-06-01_20250206_143022.csv`.

Ese CSV contiene, por contrato, el puntaje de riesgo (probabilidad de fraude/anomalía) según el modelo. Si en la configuración está definido **output_columns**, el archivo incluirá además esas columnas (por ejemplo categoría, estrato, municipio); si no, solo aparecen **contrato** y **score**.

Varios archivos en la carpeta: cada ejecución genera un archivo nuevo. El más reciente (por fecha en el nombre) corresponde a la última vez que ejecutó la inferencia.

Logs de ejecución

Los scripts guardan un registro detallado de cada ejecución en `data/logs/`:

- `etl_YYYYMMDD_HHMMSS.log` – log del ETL.
- `inference_YYYYMMDD_HHMMSS.log` – log de la inferencia.
- `train_YYYYMMDD_HHMMSS.log` – log del entrenamiento (si aplica).

Revise estos archivos si necesita más detalle sobre qué se procesó o dónde ocurrió un error.

6. Ejecución alternativa: notebooks

Si prefiere usar **Jupyter/JupyterLab** en lugar de la terminal, puede ejecutar el flujo con los notebooks en la carpeta `poc/`:

Paso	Notebook	Qué hace
------	----------	----------

Paso	Notebook	Qué hace
1. ETL	poc/1_etl.ipynb	Lee data/raw/, procesa y escribe en data/interim/.
2. Inferencia	poc/inference.ipynb	Construye dataset, carga modelo y genera scores en data/predictions/.

Importante: Los notebooks también leen la configuración de config/config.yaml, así que primero edite el **cutoff** (y **columns_filter** si aplica) antes de ejecutarlos.

Para usar los notebooks:

1. Abra JupyterLab o Jupyter Notebook desde la **raíz del proyecto**.
2. Navegue a poc/ y abra el notebook correspondiente.
3. Ejecute todas las celdas en orden (menú Run → Run All Cells o Shift+Enter celda por celda).

7. Si algo falla

- **Error al cargar la config:** Compruebe que config/config.yaml existe y que la fecha en inference.cutoff tiene formato YYYY-MM-DD y está entre comillas.
- **Error “inference.cutoff es obligatorio”:** No deje el cutoff vacío ni en blanco. Ponga una fecha válida, por ejemplo "2026-03-01".
- **Error por “meses cargados” o datos insuficientes:** Asegúrese de haber ejecutado el ETL para los meses que necesita la ventana de consumo (por defecto 12 meses hacia atrás desde el cutoff). Debe haber archivos en data/interim/consumo/ para esos meses.
- **Error al cargar el modelo:** Verifique que en models/ existan lgbm_model.pkl y features.pkl. Si faltan, debe proporcionarlos el equipo que entrena el modelo.
- **Varios archivos scores_... en data/predictions:** Es normal. Cada ejecución genera un archivo con fecha y hora en el nombre. El más reciente es el de la última ejecución.

Si el mensaje de error no le resulta claro, anote el texto completo del error y contacte al equipo técnico.

8. Entrenamiento del modelo (no es mensual)

El **entrenamiento** del modelo (python scripts/run_train.py) no forma parte del flujo mensual del usuario. Se ejecuta cuando el equipo de análisis decide reentrenar (por ejemplo con más datos o nuevos parámetros). Usted solo debe ejecutar **ETL** (si hay datos nuevos) e **inferencia** cada mes, después de actualizar **inference.cutoff** en config/config.yaml.

Resumen rápido

Qué hacer cada mes	Dónde / Cómo
1. Indicar el mes a predecir	Editar config/config.yaml → inference.cutoff (formato "YYYY-MM-DD")

Qué hacer cada mes	Dónde / Cómo
2. (Opcional) Filtrar por categoría/ciclo	Editar <code>config/config.yaml</code> → <code>inference.columns_filter</code> (o dejar <code>null</code>)
3. Cargar datos nuevos (si aplica)	Colocar archivos en <code>data/raw/inspecciones/</code> y <code>data/raw/consumo/</code>
4. Ejecutar ETL (si hay datos nuevos)	<code>python scripts/run_etl.py</code> o notebook <code>poc/1_etl.ipynb</code>
5. Ejecutar inferencia	<code>python scripts/run_inference.py</code> o notebook <code>poc/inference.ipynb</code>
6. Revisar resultados	Abrir el archivo en <code>data/predictions/</code> (nombre tipo <code>scores_<CUTOFF>_<timestamp>.csv</code> ; el más reciente es el último)
7. (Opcional) Revisar logs	Ver archivos en <code>data/logs/</code>